

Model Predictive Control

Chapter 12: Summary, Extensions & Remarks

Prof. Melanie Zeilinger

ETH Zurich

Spring 2026

Coauthors: Dr. Andrea Carron
Prof. Johannes Köhler, Imperial College London

Outline

1. Summary

2. Extensions & Remarks

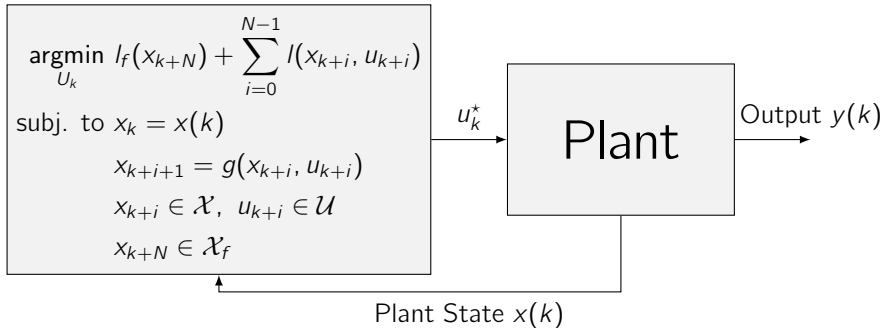
Goal: Optimal Constrained Control

$$x(k+1) = g(x(k), u(k)) \quad x, u \in \mathcal{X}, \mathcal{U}$$

Design control law $u(k) = \kappa(x(k))$ such that the system:

1. Satisfies constraints : $\{x(k)\} \subset \mathcal{X}, \{u(k)\} \subset \mathcal{U}$
2. Is stable: $\lim_{k \rightarrow \infty} x(k) = 0$
3. Optimizes “performance”
4. Maximizes the set $\{x(0) \mid \text{Conditions 1-3 are met}\}$

MPC: Receding Horizon Control



At each sample time:

- Measure / estimate current state $x(k)$
- Find the optimal input sequence for the entire planning window N :
$$U_k^* = \{u_k^*, u_{k+1}^*, \dots, u_{k+N-1}^*\}$$
- Implement only the first control action u_k^*

Important aspects of Model Predictive Control

Main advantages:

- Systematic approach for handling **constraints**
- High **performance** controller

Main challenges:

- **Feasibility:**
Optimization problem may become infeasible at some future time step, i.e. there does not exist a plan satisfying all constraints
- **Stability:**
Closed-loop stability, i.e. convergence, is not guaranteed
- **Robustness:**
The closed-loop system is not robust against uncertainties or disturbances
- **Implementation:**
MPC problem has to be solved in real-time, i.e. within the sampling time of the system, and with available hardware (storage, processor,...).

2: Unconstrained Optimal Control Summary

- Learn to compute finite horizon unconstrained linear quadratic optimal controller
 1. **Batch Approach**, which yields a series of **numerical values** for the input
 2. **Recursive Approach**, which uses Dynamic Programming to compute control **policies** or **laws**, i.e. functions that describe how the control decisions depend on the system states.
- Understand principle of optimality

$$J_j^*(x_j) = \min_{u_j} l(x_j, u_j) + J_{j+1}^*(x_{j+1})$$
$$\text{subj. to } x_{j+1} = Ax_j + Bu_j$$

- Learn to compute infinite horizon unconstrained LQR
 - Discrete-time algebraic Riccati equation
- Understand impact of horizon length in finite horizon optimal control
- Learn how to 'simulate' quasi-infinite horizon
 - Choose terminal weight via infinite horizon cost (or approximation)

3: Optimization

- Understand property of convexity of sets and functions, define convex optimization problem
- Understand benefit of convex optimization problems
 - Any locally optimal solution is globally optimal
- Learn and contrast properties of LPs and QPs
- Pose the dual problem to a given primal optimization problem
 - Lagrangian → Dual function
- Test optimality of a primal and dual solution by means of KKT conditions
 - Stationarity, primal & dual feasibility, complementarity slackness
- Understand meaning of dual solution for the cost function
 - Active constraints, sensitivity of cost w.r.t. constraints

4: Constrained Finite Time Optimal Control

- Understand feasible set of constrained finite horizon optimal control (CFTOC) problem

$$\mathcal{X}_N = \{x_0 \in \mathbb{R}^n \mid \exists (u_0, \dots, u_{N-1}) \text{ such that } x_i \in \mathcal{X}, u_i \in \mathcal{U}, \\ i = 0, \dots, N-1, x_N \in \mathcal{X}_f, \text{ where } x_{i+1} = Ax_i + Bu_i\}$$

- Write quadratic cost CFTOC as QP
- Write $1-\infty$ -norm cost CFTOC as LP
- Contrast properties of LP and QP solution
- Understand structure of the optimal MPC control law
→ piece-wise affine control law defined over polyhedra (critical regions)

5: Invariance

- Learn definition and meaning of **invariance**
 - Region in which an **autonomous** system remains **for all time**
- Learn definition and meaning of **control invariance**
 - Region for which there **exists a controller** so that the system remains within region **for all time**
- Learn how to (conceptually) compute these sets
 - invariant set iff $\mathcal{O} \subseteq \text{pre}(\mathcal{O})$
- Learn how to compute polytopic and ellipsoidal invariant sets

6: Recursive Feasibility and Stability

- Prove stability of nonlinear system via Lyapunov function
- Contrast stability properties of LQR and MPC for constrained problems
- Understand why MPC by itself does not provide guarantees on stability and constraint satisfaction
 - short-sidedness of finite horizon control causes deviation between the open-loop prediction and the closed-loop system
- Design MPC with closed-loop stability and constraint satisfaction guarantees
 - State sufficient conditions
 - Engineer terminal ingredients
 - Understand main proof idea

7: Practical

- Formulate MPC tracking problem
→ coordinate transformation, translate constraints, scale terminal set
- Address constant disturbances to achieve offset-free tracking/regulation
→ estimate disturbance, adjust steady-state to account for error
- Formulate soft-constrained MPC problem and understand its properties

8/9: Robust MPC

- Understand concept of robust invariance, learn algorithm to compute robust invariant sets
- Reformulate constraints to ensure robust constraint satisfaction
- Formulate robust open-loop MPC problem
- Understand benefit of closed-loop predictions in presence of uncertainties
- Understand and contrast concepts of constraint-tightening and tube MPC
- Design tube MPC problem
- Prove robust constraint satisfaction and robust stability of tube MPC

10: Implementation

- Understand principle of computing solution to parametric optimization problem
- Understand challenges of explicit MPC solution
→ exponential growth of complexity for computation & point location
- Learn principles of gradient descent and Newton's method for unconstrained minimization
- Learn extension to constrained minimization via projected gradient descent and interior-point method

MPC – What we have learned

- Nominal MPC
 - Constraint satisfaction and invariance
 - Feasibility & stability guarantees
 - Translation into convex optimization problem, solution properties
- (Offset-free) Tracking MPC: Tracking of non-zero setpoint, compensation of constant disturbance
- Soft-constrained MPC: Feasibility through constraint relaxation
- Robust MPC: Constraint satisfaction despite bounded disturbances

⇒ Systematic approach for constraint satisfaction (the most successful one).

⇒ This class: For linear systems, potentially bounded uncertainties

⇒ Intuitive and flexible formulation: In principle any model and any cost function (but theory difficult).

Outline

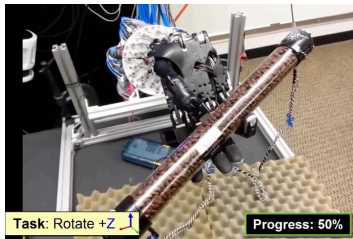
1. Summary

2. Extensions & Remarks

Challenges

Which problems can we not address with basic MPC yet?

Following complex references / goals



[ADROIT Manipulation Platform]

<https://www.youtube.com/watch?v=IN7VFquI7ow>



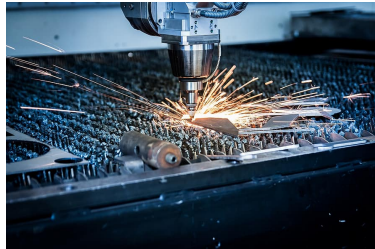
[Boston Dynamics] <https://www.youtube.com/watch?v=kHBcVlqpVZ8>

Optimizing a true cost

Example: Process control
Maximize yield, efficiency,



Example: Manufacturing
Maximize quality



Uncertainties and variation



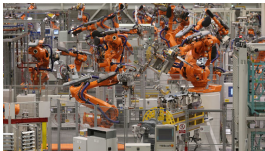
Large-scale systems of systems



Horns Rev Wind Farm, Jha et. al, Wind Energy Symposium 2018



Future power grid (Photograph: Krunja / Dreamstime.com)



Factory (Credit: Oli Scarff/Getty Images)



Satellite formation flight (Credit: DARPA artist's concept)

Towards complex systems and tasks

- Complex dynamical systems
- More flexible cost functions
- Account for uncertainty and variation
- Reduce uncertainty
- Address control of multiple interacting systems

Theory:

- Nonlinear MPC
- Economic MPC
- Robust & Stochastic MPC
- Learning-based MPC
- Distributed MPC
-

We will give a brief outlook on some of these in the in the following...

Outline

2. Extensions & Remarks

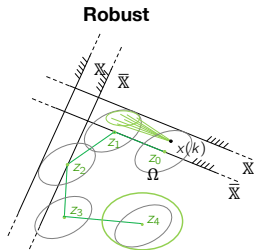
MPC under Uncertainty

Hybrid Systems

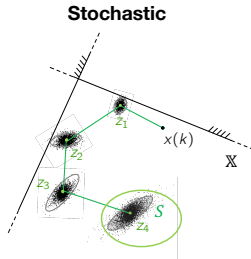
Nonlinear Systems

Learning & MPC

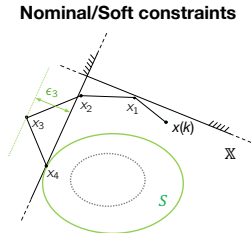
MPC Under Uncertainty



- Consider worst-case disturbances
- Robust constraint satisfaction $\forall w \in \mathcal{W}$
- “Robust stability”



- Consider disturbances according to (i.i.d.) distribution
- Constraint satisfaction in probability for $w \sim \mathcal{Q}^w$
- “Stochastic stability”



- No explicit uncertainty treatment
- (Potentially) Constraint relaxation
- “Robust stability”
 $\forall w \in \mathcal{W}$

Uncertainty characterization in robust control/MPC

Sources of model mismatch encountered in applications (e.g., autonomous driving car)

- Unpredictable disturbances (e.g., wind)
- Measurement noise (e.g., quantization error in sensor)
- Parametric uncertainty (e.g., friction coefficient not exactly estimated)
- Unmodelled/simplified nonlinearities (e.g., simplified linear friction model)
- Dynamic uncertainty (e.g., lower order model ignoring actuator dynamics)
- ...

⇒ Bounding all model-mismatch with an additive bounded disturbance $w(k) \in \mathcal{W}$ is simple,
but can be quite conservative. . .

Example: Robust Model with bounded Parametric Uncertainty

System: $x(k+1) = f(x(k), u(k), \theta_t) + \tilde{w}(k), \tilde{w} \in \tilde{\mathcal{W}}$

θ_t constant but unknown

→ Assume parametric uncertainty $\theta \in \Theta$

Model: $x(k+1) = f(x(k), u(k), \hat{\theta}) + w(k)$

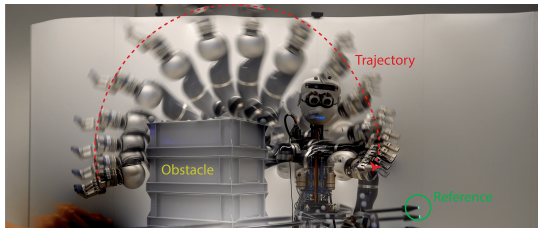
Assumption: $w(k) \in \mathcal{W} \forall k$

$$\Rightarrow \mathcal{W} = \left\{ w \in \mathbb{R}^n \mid w = f(x, u, \theta) + \tilde{w} - f(x, u, \hat{\theta}), \right. \\ \left. \theta \in \Theta, x \in \mathcal{X}, u \in \mathcal{U}, \tilde{w} \in \tilde{\mathcal{W}} \right\}.$$

⇒ Robust treatment of $w \in \mathcal{W}$

Uncertainty characterization in robust MPC

Example – robotic manipulator¹

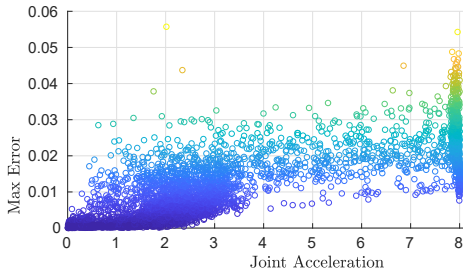


- Robotic manipulator
- Sources of model mismatch:
 - noisy measurements;
 - estimated inertia matrix;
 - unmodelled actuator dynamics;
 - ...

¹Julian Nubert, Johannes Köhler, Vincent Berenz, Frank Allgöwer, and Sebastian Trimpe. Safe and fast tracking on a robot manipulator: Robust MPC and neural network control. IEEE Robotics and Automation Letters, 5(2):3050–3057, 2020.

Uncertainty characterization in robust MPC

Example – robotic manipulator¹



- Disturbance magnitude highly correlated to acceleration
- Constant disturbance bound might be conservative
- Incorporate the knowledge in the design

¹Julian Nubert, Johannes Köhler, Vincent Berenz, Frank Allgöwer, and Sebastian Trimpe. Safe and fast tracking on a robot manipulator: Robust MPC and neural network control. IEEE Robotics and Automation Letters, 5(2):3050-3057, 2020.

Uncertainty characterization in robust MPC

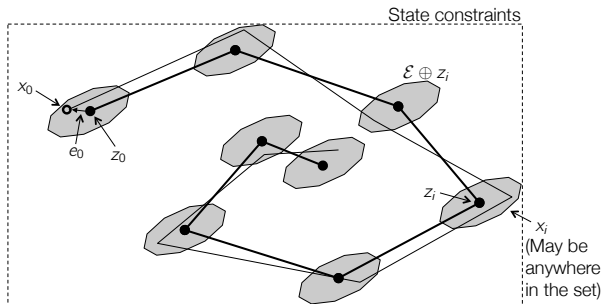
- State and input dependent disturbance bound $w(k) \in \mathcal{W}(x(k), u(k))$:

$$\mathcal{W}(x, u) = \{w = \Delta A \cdot x + \Delta B \cdot u + d, (\Delta A, \Delta B) \in \Omega, d \in \mathcal{D}\},$$

with known polytopes $\Omega, \mathcal{D}$.

- How can we use this knowledge to enforce robust constraint satisfaction?

Main Idea of Tube MPC



Plan the nominal trajectory such that all possible state trajectories satisfy the **state constraint**.

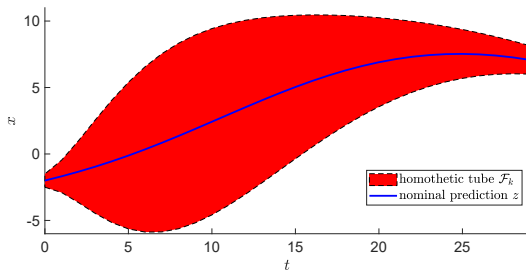
Homothetic tube

- Tube parametrization:

$$\mathcal{F}_k = \{e \mid V(e) \leq \delta_k\}$$

using an (offline designed) Lyapunov function $V(x) = \|x\|_M$, error $e = x - z_k$, a scaling $\delta_k > 0$, and a positive definite matrix M .

- Predict tube-size δ_k based on the nominal trajectory z , v and the disturbance bound $\mathcal{W}(x, u)$.



Outline

2. Extensions & Remarks

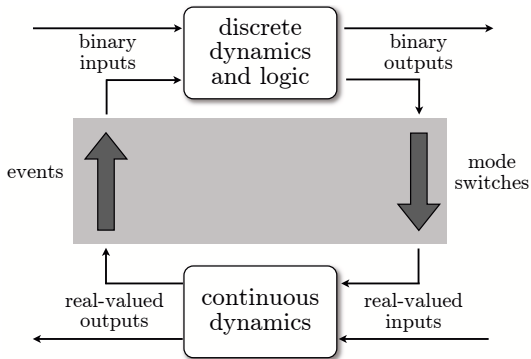
MPC under Uncertainty

Hybrid Systems

Nonlinear Systems

Learning & MPC

Special Class: Hybrid Systems



Hybrid systems: Logic-based discrete dynamics and continuous dynamics interact through events and mode switches

Piecewise Affine (PWA) Systems

PWA systems are defined by:

- **affine dynamics and output** in each region:

$$\begin{cases} x(k+1) &= A_j x(k) + B_j u(k) + f_j \\ y(k) &= C_j x(k) + D_j u(k) + g_j \end{cases} \text{ if } (x(k), u(k)) \in \mathcal{X}_j$$

- **polyhedral partition** of the (x, u) -space:

$$\{\mathcal{X}_j\}_{j=1}^S := \{x, u \mid H_j x + J_j u \leq K_j\}$$

with $x \in \mathbb{R}^n$, $u \in \mathbb{R}^m$

Physical constraints on $x(k)$ and $u(k)$ are defined by polyhedra $\mathcal{X}_j$

Piecewise Affine (PWA) Systems

Examples:

- *linearization* of a non-linear system at different operating point
⇒ useful as an approximation tool
- *closed-loop MPC system* for linear constrained systems
- When the mode i is an exogenous variable, the partition disappears and we refer to the system as a **Switched Affine System (SAS)**

Mixed Logical Dynamical Systems

Goal: Describe hybrid system in form compatible with optimization software:

- continuous and binary variables
- linear equalities and inequalities

Idea: associate to each Boolean variable p_j a binary integer variable δ_j :

$$p_j \Leftrightarrow \{\delta_j = 1\}, \quad \neg p_j \Leftrightarrow \{\delta_j = 0\}$$

and embed them into a set of constraints as **linear integer inequalities**.

Two main steps:

1. Translation of Logic Rules into Linear Integer Inequalities
2. Translation of continuous and logical components into Linear Mixed-Integer Relations

Final result: a compact model with linear equalities and inequalities involving real and binary variables

Example: “IF-THEN-ELSE” Relation

$$\begin{array}{ll} \text{IF } p & \text{THEN} \\ & z_k = a_1^T x_k + b_1 \\ \text{ELSE} & \\ & z_k = a_2^T x_k + b_2 \end{array} \iff \begin{array}{ll} (m_2 - M_1)\delta + z_k & \leq a_2^T x_k + b_2 \\ (m_1 - M_2)\delta - z_k & \leq -a_2^T x_k - b_2 \\ (m_1 - M_2)(1 - \delta) + z_k & \leq a_1^T x_k + b_1 \\ (m_2 - M_1)(1 - \delta) - z_k & \leq -a_1^T x_k - b_1 \end{array}$$

where $x \in \mathcal{X}$, with

$$\sup_{x \in \mathcal{X}} a_j^T x + b_j \leq M_j,$$

$$\inf_{x \in \mathcal{X}} a_j^T x + b_j \geq m_j.$$

MPC for Hybrid Systems: General Formulation

Consider the CFTOC problem:

$$\begin{aligned} J^*(x(k)) = \min_U & l_f(x_N) + \sum_{i=0}^{N-1} l(x_i, u_i, \delta_i, z_i) \\ \text{subj. to } & x_{i+1} = Ax_i + B_1 u_i + B_2 \delta_i + B_3 z_i \\ & E_2 \delta_i + E_3 z_i \leq E_4 x_i + E_1 u_i + E_5 \\ & x_N \in \mathcal{X}_f \\ & x_0 = x(k) \end{aligned}$$

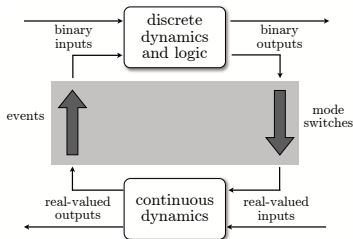
where $x \in \mathbb{R}^{n_c} \times \{0, 1\}^{n_b}$, $u \in \mathbb{R}^{m_c} \times \{0, 1\}^{m_b}$, $y \in \mathbb{R}^{p_c} \times \{0, 1\}^{p_b}$, $\delta \in \{0, 1\}^{r_b}$ and $z \in \mathbb{R}^{r_c}$ and

$$U = \{u_0, u_1, \dots, u_{N-1}\}$$

Mixed Integer Optimization

Summary: Hybrid Systems

- Hybrid systems: mixture of continuous and discrete dynamics



Logic-based discrete dynamics and continuous dynamics interact through events and mode switches

- Many important systems fall in this class
- Many tricks involved in modeling - automatic systems available to convert to consistent form (e.g. HYSDEL)
- Optimization problem becomes a mixed-integer linear / quadratic program
 - NP-hard (exponential time to solve)
 - Advanced commercial solvers available
- MPC theory (invariance, stability, etc) applies
 - Computing invariant sets is usually extremely difficult
 - Computing the optimal solution is extremely difficult (sub-optimal ok)

Outline

2. Extensions & Remarks

MPC under Uncertainty

Hybrid Systems

Nonlinear Systems

Learning & MPC

Nonlinear MPC

Consider the nonlinear system dynamics: $x(k+1) = g(x(k), u(k))$

$$\begin{aligned} J^*(x(k)) = \min_U \quad & l_f(x_N) + \sum_{i=0}^{N-1} l(x_i, u_i) \\ \text{subj. to} \quad & x_{i+1} = g(x_i, u_i), \quad i = 0, \dots, N-1 \\ & x_i \in \mathcal{X}, \quad u_i \in \mathcal{U}, \quad i = 0, \dots, N-1 \\ & x_N \in \mathcal{X}_f \\ & x_0 = x(k) \end{aligned}$$

- Presented assumptions on the terminal set and cost did not rely on linearity
- Lyapunov stability is a general framework to analyze stability of nonlinear dynamic systems

→ Results can be directly extended to nonlinear systems.

However, computing the sets $\mathcal{X}_f$ and function l_f can be very difficult!

Feasibility and Stability of Nonlinear MPC

Practical approaches:

- Choose zero terminal constraint (no terminal cost needed)
- Linearization² (for quadratic cost)
 - Linearize system around origin, $x^l(k+1) = A^l x(k) + B^l u(k)$, assume linearization is stabilizable.
 - Design auxiliary controller $\kappa_f(x) = Kx$, terminal cost $l_f(x) = x^T P x$ and constraint set $\mathcal{X}_f = \{x | x^T P x \leq \alpha\}$ for linearized system, s.t.
 - $l_f(A^l + B^l K)x - l_f(x) = -2x^T(Q + K^T R K)x \quad \forall x \in \mathcal{X}_f$
 - All state and input constraints are satisfied in $\mathcal{X}_f$
 - α is small enough such that

$$\begin{aligned} l_f(g(x, Kx)) - l_f((A^l + B^l K)x) &\leq x^T(Q + K^T R K)x & \forall x \in \mathcal{X}_f \\ \Rightarrow l_f(g(x, Kx)) - l_f(x) &\leq -x^T(Q + K^T R K)x & \forall x \in \mathcal{X}_f \end{aligned}$$

Terminal cost is a Lyapunov function in the terminal set and terminal set is invariant also for the nonlinear system.

²Model Predictive Control: Theory and Design, J. B. Rawlings and D. Q. Mayne, 2009.

Remarks on Robust Nonlinear MPC

The "error dynamics" for nonlinear systems are given by

$$\begin{aligned}e_{i+1} &= x_{i+1} - z_{i+1} \\&= f(x_i, u_i) + w_i - f(z_i, v_i) \\&= f(z_i + e_i, \kappa(z_i + e_i, z_i, v_i)) - f(z_i, v_i) + w_i \\&=: g(e_i, z_i, v_i) + w_i \neq g(e_i) + w_i.\end{aligned}$$

⇒ Dynamics of the error also depend on nominal state and input z, v

⇒ Exact disturbance propagation is difficult for nonlinear systems.

⇒ Need a different tool to ensure that the true system stays close to the nominal system (incremental stability).

Nonlinear MPC Computation

- At each time step: Linearize system around a trajectory (usually solution from previous time step). Solve convex problem.
 - No theoretical guarantees, but often applied in practice.
- Solve nonlinear program:
 - Interior-point method.
Solvers: IPOPT, FORCES Pro, KNITRO
 - Sequential quadratic programming
Solvers: SNOPT, ACADO, NPSOL, KNITRO
- Non-convex problem, convergence only to locally optimal solution (under some assumptions).

Remarks on Sequential Quadratic Programming

$$\begin{aligned} \min_z \quad & f(z) \\ \text{subj. to} \quad & g_i(z) \leq 0, i = 1, \dots, m \\ & h_i(z) = 0, i = 1, \dots, p \end{aligned}$$

Idea: Approximate via iterative solution of quadratic approximation

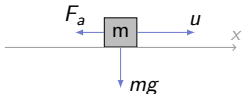
At iterate (x_k, λ_k, ν_k) solve

$$\begin{aligned} \min_p \quad & \frac{1}{2} p^T W_k p + \nabla f_k^T p \\ \text{subj. to} \quad & \nabla g_i(z_k)^T p + g_i(z_k) \leq 0, i = 1, \dots, m \\ & \nabla h_i(z_k)^T p + h_i(z_k) = 0, i = 1, \dots, p \end{aligned}$$

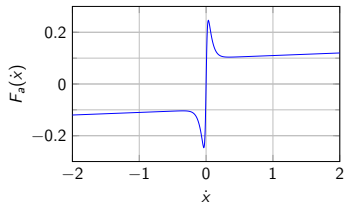
with $W_k = \nabla^2 \mathcal{L}(z_k, \lambda_k, \nu_k)$ the Hessian of the Lagrangian function.

$\Rightarrow$ Real-time iteration: Run one inner iteration, i.e. of the QP solution

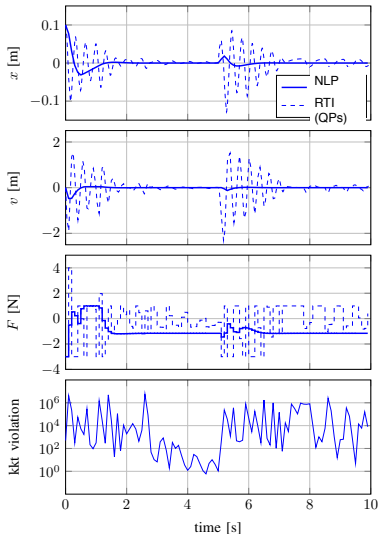
Nonlinear MPC Computation: Example



$$-m\ddot{x} + u - mgF_a(\dot{x}) = 0$$



$$F_a(\dot{x}) = 0.25(\tanh(100\dot{x}) - \tanh(10\dot{x})) \\ + 0.1 \tanh(50\dot{x}) + 0.01\dot{x}$$



Outline

2. Extensions & Remarks

MPC under Uncertainty

Hybrid Systems

Nonlinear Systems

Learning & MPC

Back to the start: Model Predictive Control vs. Reinforcement Learning



Yann LeCun ✓
@ylecun

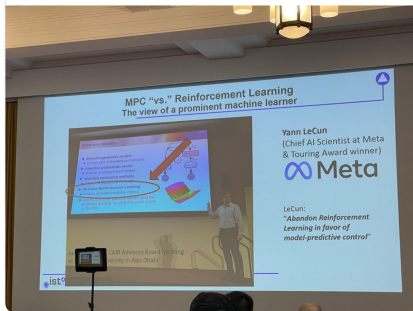
...

Indeed, I do favor MPC over RL.
I've been making that point since at least 2016.
RL requires ridiculously large numbers of trials to learn any new task.
In contrast MPC is zero shot: If you have a good world model and a good task objective, MPC can solve new tasks without any task-specific learning.
That's the magic of planning.
It doesn't mean that RL is useless, but its use should be a last resort.



Antoine Leeman @antoine_leeman · 24 Aug 2024

Yann Lecun favours MPC over RL? Interesting! What about data-augmented MPC? @ylecun #nmpc2024



9:17 pm · 25 Aug 2024 · 265.5K Views



Dimitri Bertsekas · 1st
Fulton Professor at School of Computing and Augmented Intelligence, Ar...
2mo · Edited ·

...

I recently came across the statement "Abandon Reinforcement Learning in favor of model predictive control," by Turing Award winner and deep learning pioneer Yann LeCun from one of his video lectures ("Objective-Driven AI" talk at the University of Washington, 2024). I found it remarkable and courageous, as it challenges the prevailing orthodoxy - at least outside the decision and control community. LeCun expands on this view in several public lectures.

The statement resonated with me, as I have expressed a related perspective in my books and in my course at ASU since 2019.

In summary: "MPC is part of RL, and reversely, the most reliable part of RL is MPC". The key points are:

A. Reinforcement learning (RL) and (approximate) dynamic programming (DP) address the same mathematical problem - optimal and suboptimal sequential decision making - and revolve around two central objects: value functions and policies.

Remarks on RL & MPC

Two approximations of the “true” optimal control problem

Model Predictive Control:

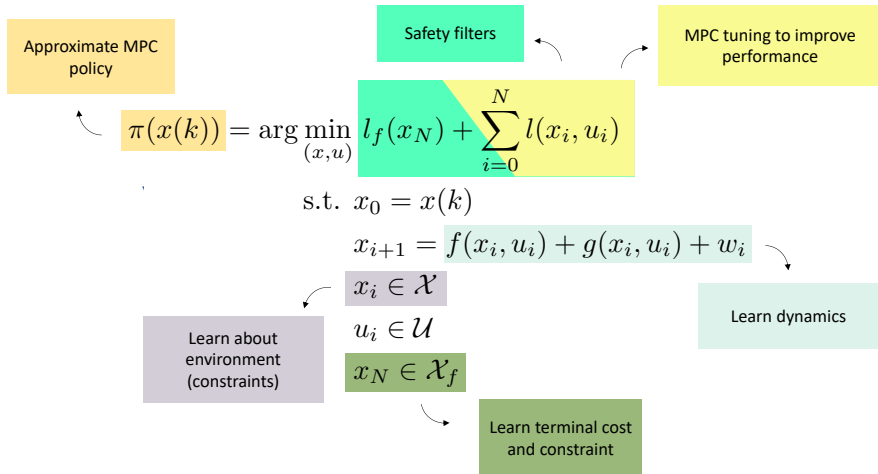
- Approximate objective, model, constraints
- Solve optimization problem numerically (in receding horizon)

Reinforcement learning:

- Evaluate objective empirically (typically in episodes)
- Iterative improvements (often model-free, no constraints)

⇒ Many powerful combinations of learning and MPC

Use of Learning in MPC – Overview



Upcoming Courses

- Large-scale Convex Optimization, Block Course 1.-12.6.2026
- Game Theory and Control, Fall semester 2026
- Advanced MPC, Spring semester 2027 (tentative)
(Focus on Learning-based MPC)